



GRIFFITH COLLEGE DUBLIN
ASSIGNMENT TITLE SHEET
FACULTY OF COMPUTING SCIENCE

Course:	BSC
Stage/Year:	3
Module:	WT
Semester:	Semester 2
Assignment Number:	Assignment 3
Date of Title Issue:	08/04/26
Assignment Deadline:	27/04/26 by 11:59pm
Assignment Submission:	Upload to Moodle
Assignment Weighting:	30% of module

Assignment Title

WT – Assignment 3 - Client-side components

Artificial Intelligence Assessment Scale (AI AS):

This assignment is subject to AI permission option: (select all that apply)

No AI Use	☐
AI for Planning	☐
AI for Editing	☒
AI for Support Tasks	☐
AI for Collaboration	☐
Full Use of AI	☐

AI Assessment Scale Framework: <https://www.griffith.ie/sites/default/files/2025-05/f-6.5-ai-in-learner-assessment-policy.pdf>

Please note:

- All AI generated content and process must be declared **and** screenshots of both prompts and responses must be included in an appendix.
- Any task that is not listed as permitted may be assumed to be prohibited.
- If it is determined that a learner has used an AI tool in any way that is not permitted by the chosen level, then they will be subject to the disciplinary procedures listed in the Academic Integrity and Misconduct policy regarding the misuse of AI in assignments.

Assignment Specific AI Instructions:

- AI may be used in **support tasks only** – eg. using the IDE'S AI assistant to generate in-place code suggestions.
- Generative AI may **not** be used to create blocks of code or whole sections of assignments.
- Where generative AI such as ChatGPT has been used for tutorial purposes, links must be provided to the chat along with screenshots of both prompts and responses included in an documentation appendix.
- **No AI may be used to write reports, documentation, or comments, and may not be used to create images, charts/graphs, wireframes, sitemaps, etc.**
- You may be called in for VIVA to explain your work if unauthorized/uncredited AI use is suspected.

Learning Outcomes

This assignment is assessing the following programme and related module learning outcomes:

LO3: Evaluate the architecture, System Analysis, design and features of client-side web application frameworks

LO5: Install and configure client and server-side web application frameworks and single-page applications

LO6: Plan, design and build web applications using both client-side and server-side web application frameworks.

Submission Instructions and Plagiarism

Plagiarism:

All work must be your own. Any cases of suspected plagiarism will be thoroughly investigated and may be brought in for VIVA. All parties involved in cases of plagiarism will receive a 0 and their student record marked accordingly. Any submission that is found to be plagiarised will receive a grade of 0 without the possibility of resubmission.

By submitting your assignment you accept that you understand what plagiarism is and that your assignment is not plagiarised in any way.

You may not submit work or partial work that has previously been submitted for assessment.

Instructions for submission:

Upload your report as a **.zip** to Moodle. Each team submits **only one copy**.

Submission naming convention: Save your zip file with the name of your project (only .zip is accepted). Be sure include all members' names in the coversheet. See below for details of exact structure

Penalties

Late Penalties:

Standard penalties will be applied to work that is submitted late, as per faculty guidelines.
<https://moodle.griffith.ie/mod/resource/view.php?id=18202>

Penalties that apply to this assignment:

- 100 marks - No demo (**no points will be awarded if you do not demo your work**)
- 10 marks - Your team did not submit all files necessary for your project to be functioning
- 10 marks - Your team did not submit a URL to your app hosted online.

Project Overview

In this assignment, you will work in your project groups to build client-side user interface of the project you proposed in Assignment 1 using React.js, a client-side JavaScript framework. The goal of this assignment is for you to create a user interface for the application you propose in assignment 1.

For the purposes of this assignment, you'll be making at least **three** different views/pages that will be part of your project. As an example, you may choose to create the landing page, login page, and some data entry pages. Once you are satisfied with your pages, you will implement the logic for the client.

You are encouraged to add enough dynamic behaviour to your application to fulfil the vision that you set out in your project proposal. Please consider the following as minimum requirements only—you are encouraged to get creative and do more than this, but this is what will be required to receive pass marks.

Project requirements:

These requirements must be implemented in **React.js**:

- Create at least 3 views/pages that best represent functionalities you proposed – label these clearly in the comments. No HTTP requests should actually be made, but it should be clear what elements will eventually map to functionality on the backend.
- The views must allow users to interact with the system – users should be able to add input, use buttons and links etc.
- You decide the kinds of elements (for data entries) to implement that are appropriate to your project.
- Appropriate CSS or CSS libraries should be used for styling, and help support the user experience.
- Perform client-side input validation.
- Your code must be written with JavaScript coding standards in mind.

- Support responsive layouts (i.e., support different devices, different sizes of screens).
- Your code must be written with JavaScript coding standards in mind.
- Overall, your system must be functioning. Your entire project must work as expected; it must provide normal behaviour, given the domain application of the project.

Coding Standards

- Use good coding style
- Make identifier names understandable
- Use proper and consistent indentation
- Use comments. All code must be commented.

Deploy Online

- Deploy your system online with <https://render.com/>
- Submit a URL to your app hosted online along with your source code.

Required outputs:

What to include

- All of your project files – see guidelines on Moodle for more detail.
The files submitted should be **all** the files necessary for your project to run.
- A readme.md file detailing:
 - your work so far and the contributions of each team member to this iteration of the project.
 - Include all members' names as authors of the programs/files based on contribution.
 - References to any tutorials, guides, third-party resources etc.
- A coversheet with all contributing students' full names and student numbers.

How to submit

- Save your site as a **ZIP file with the following name <groupname><assignment3><AppName>.zip** (only ZIP is accepted!)
- Be sure include all members' names in the coversheet.
- Each team submits only **one** copy.
- Put your site information readme.md and cover sheet as a PDF in your site root folder.
- Upload your site as a ZIP file to **Moodle**

Division of Labour

All team members are expected to contribute equally to the project. At each iteration the team will fill state how the work was divided amongst the team.

E.g. If the work was split evenly per team member, then that is fine and simply say “Work was evenly divided”. If this was not the case, then state with a summary sentence or percentage.

This should be included in your README like:

John: Backend routes (40%)

Sarah: Database + models (30%)

Emma: Testing + documentation (30%)

Grading

	Components	Total Marks
General Functionality	Create the different views with respect to the functionalities you proposed	20
	Allow users to interact with the system (information architecture)	5
	Accept users' data entries (i.e., inputs)	5
	Create and/or update the style or layout of your views in response to user interaction	5
	Support responsive design / layout	5
	Appropriate use of CSS or CSS libraries for styling purposes	5
Input/Output	• Perform proper client-side input validation • Properly validate form data • Provide user-appropriate error messages • If no error message is provided, other reasonable and appropriate handling (such as auto-complete or auto-correction with defaults) and/o some forms of user-appropriate feedback are acceptable.	15
Coding style	• Use good coding style: readability, style, and maintainability • Make identifier names understandable • Use proper and consistent formatting • Use clear comments describing intended functionality	10
User demonstration	• Program is demonstrated clearly • Code segments clearly explained	10
App hosted online	Your application must be hosted online	10
Documentation	• You clearly describe your implementations in your readme document • All work is correctly cited • All work presented professionally	10
Total		100